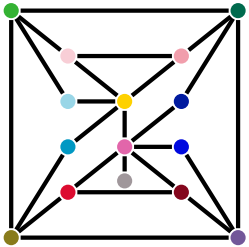
Module 7 – Project Graph Isomorphism – Instruction
Graph Isomorphism and Colour Refinement

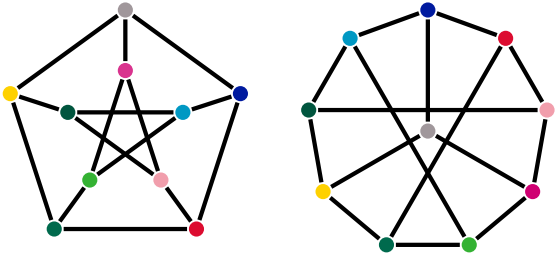


Ruben Hoeksma

Individual assignment - Today, next week, & after the break

- Today: Explanation about Graph Isomorphism (more) and the Colour Refinement Algorithm.
- Week 3: Three project sessions for assistance (questions and implementation troubles) and/or sign-off interview.
- Week 4: One more project session & Final sign-off on Wednesday (March 4).
- Sign-off in two steps:
  1. Verify correctness through CodeGrade.
  2. Interview during any of the sessions (until the deadline).
- Don't forget:
  - Individual team role inventory test (deadline today!).
  - Project Group creation.
- March 5 during lunch: Module 7/project-group activity organized by Abacus and Inter-*Actief*.

Symmetry & Similarity: Graph Isomorphism



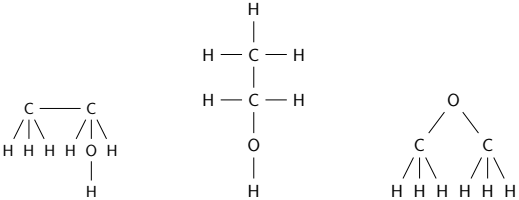
- An *isomorphism* between two simple graphs G and H is a bijection $f : V(G) \to V(H)$ such that for all $u, v \in V(G)$:

$$\{u, v\} \in E(G) \quad \Leftrightarrow \quad \{f(u), f(v)\} \in E(H)\,.$$

- Two graphs G and H are *isomorphic* if there exists an isomorphism between them.
- The *Graph Isomorphism (GI) Problem*: given two graphs G and H, are they isomorphic (or not)?

Example from Biochemistry

- Database of *molecules*, and a newly found molecule.
- Is it really new, or already in the database?



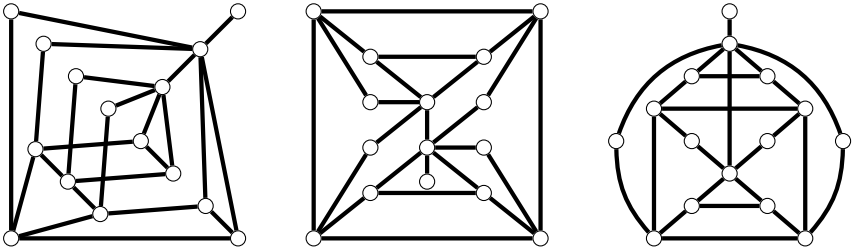
Question: But what about different atom types?

Answer:

- A *k-vertex colouring (or k-colouring)* of graph G is a function $\alpha : V(G) \to \{1, \ldots, k\}$, $\alpha(v)$ is called the *colour* of v.
  (Note: Adjacent vertices may have the same colour.)
- Given two k-colourings α and β of graphs G and H, $f : V(G) \to V(H)$ is *colour preserving* if for all $v \in V(G)$, $\alpha(v) = \beta(f(v))$.
- Colours represent different atoms; ask for *colour-preserving isomorphism*.

Total Enumeration

Which of these graphs are isomorphic?



First idea: For every graph pair, try all possible bijections, and see if there is an isomorphism among them...

Not great! $\exists$ 15! = 1.307.674.368.000 bijections. Even with fast computer, you don't want to do that! (if you verify 1.000 per sec., $>$ 40 years)
We need something smarter...

Observation 1

If $f : V(G) \to V(H)$ is isomorphism, then for every $v \in V(G)$: f yields a *bijection* from $N(v)$ to $N(f(v))$.

Proof: follows immediately from the definition of isomorphism. $\square$

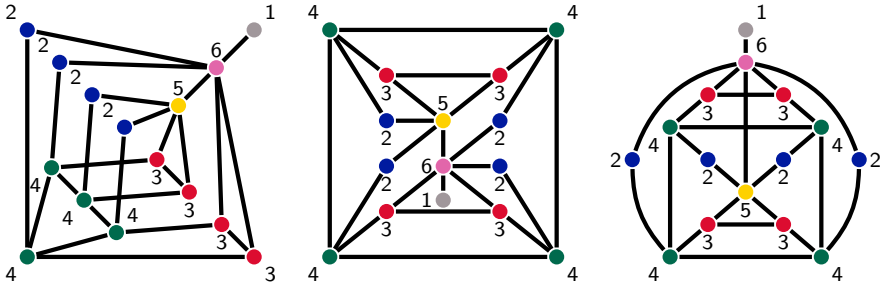
Corollary 1

If f is an isomorphism from G to H, then for every $v \in V(G)$:

$$d(v) = d(f(v)) \,.$$

(An isomorphism f is "degree-preserving")

Colouring I



2nd idea: colour all vertices v according to $d(v)$. Now, any isomorphism is *colour preserving*, so we only need to check *colour preserving bijections*.

Better: only $4! \cdot 4! \cdot 4! = 13824$ bijections need to be tested.
But can we do better?

Colouring II

As we colour G and H at the same time, consider the set of graphs $\mathcal{G} = \{G, H\}$. Denote by $V(\mathcal{G}) = V(G) \cup V(H)$ all vertices of the graphs.

Definition

Let α be a colouring of $\mathcal{G}$. Let α^c be set of vertices with colour c. $u, v \in V(\mathcal{G})$ have *differently-coloured neighbourhoods* if there exists a colour c such that $|N(u) \cap \alpha^c| \neq |N(v) \cap \alpha^c|$, and *identically coloured* otherwise.

Observation 2

Let α be a k-colouring of $\mathcal{G} = \{G, H\}$, and let f be a colour preserving isomorphism from G to H. Then for every $v \in V(G)$: v and $f(v)$ have identically coloured neighbourhoods.

Colour Refinement

Great Idea: Keep *refining* the colouring: given colouring α_i, compute α_{i+1} by assigning the same colour to two vertices u and v if and only if:

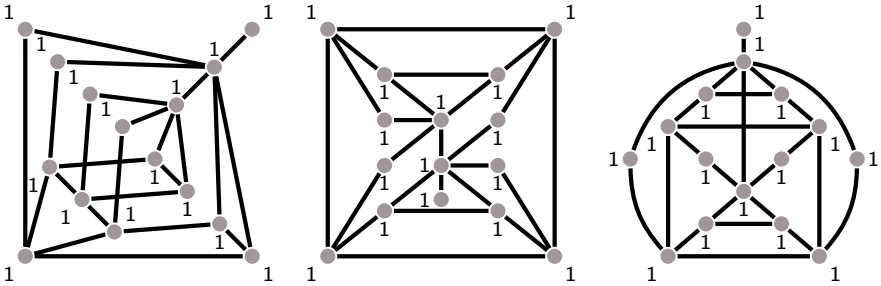
- they had the same colour before: $\alpha_i(u) = \alpha_i(v)$, and
- they had identically coloured neighbourhoods in α_i.

Remarks:

- 'Having the same colour' in α_i is an *equivalence relation*; α_{i+1} (re-)colours the *equivalence classes* of this relation.
- Key fact: the following *invariant* is maintained throughout this refining process, if it holds for the initial colouring.

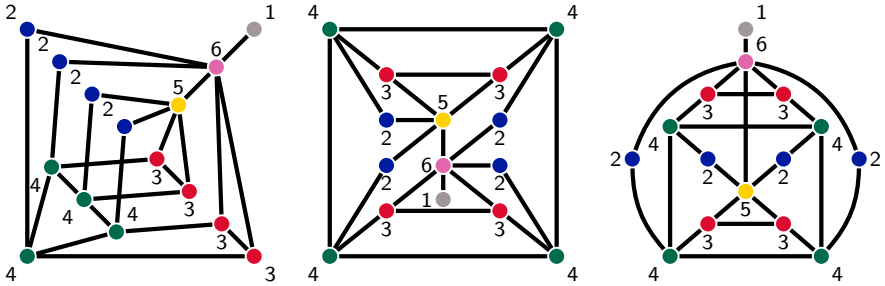
Every isomorphism f from G to H is colour preserving.

Colour Refinement: Example



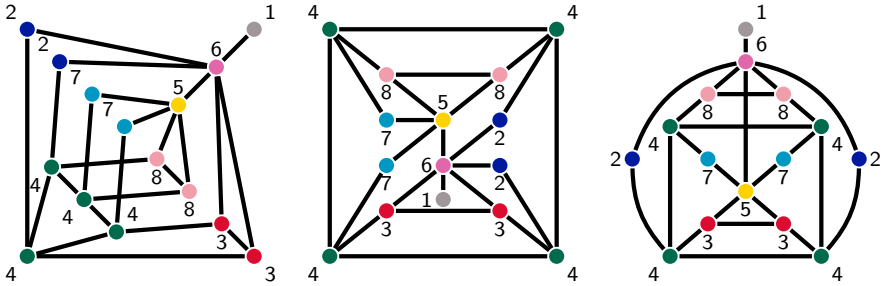
- *Colour Refinement:* Iteratively, vertices with same colour may be assigned different colours (only) if they have differently coloured neighbourhoods.
- *Invariant:* Every isomorphism is colour preserving.

Colour Refinement: Example



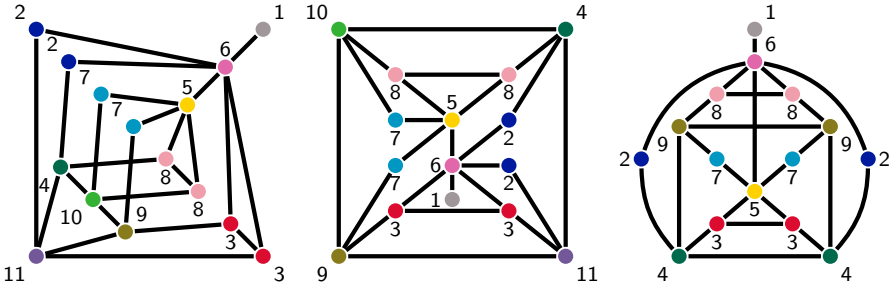
- *Colour Refinement:* Iteratively, vertices with same colour may be assigned different colours (only) if they have differently coloured neighbourhoods.
- *Invariant:* Every isomorphism is colour preserving.

Colour Refinement: Example



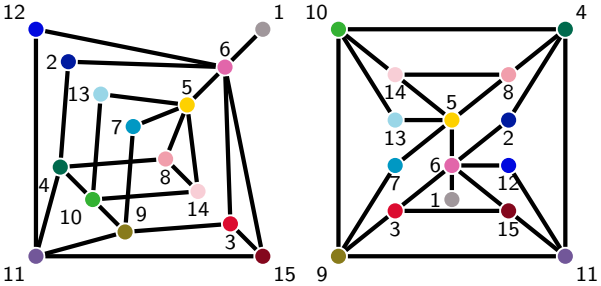
- *Colour Refinement:* Iteratively, vertices with same colour may be assigned different colours (only) if they have differently coloured neighbourhoods.
- *Invariant:* Every isomorphism is colour preserving.

Colour Refinement: Example



- *Colour Refinement:* Iteratively, vertices with same colour may be assigned different colours (only) if they have differently coloured neighbourhoods.
- *Invariant:* Every isomorphism is colour preserving.
- *Conclusion:*
  - right graph is not isomorphic to the others,
  - but the left and middle graphs are isomorphic.

Colour Refinement: Example



- *Colour Refinement:* Iteratively, vertices with same colour may be assigned different colours (only) if they have differently coloured neighbourhoods.
- *Invariant:* Every isomorphism is colour preserving.
- *Conclusion:*
  - right graph is not isomorphic to the others,
  - but the left and middle graphs are isomorphic.

Colour Refinement: Pseudocode

INPUT: A set of graphs $\mathcal{G} = \{G_1, \ldots, G_\ell\}$ *and initial colouring* α_0 *of* $\mathcal{G}$.
OUTPUT: A stable colouring α of $\mathcal{G}$.

for all $v \in V(\mathcal{G})$:
    $\alpha_0(v) := 1$
$i := 0$
While $\alpha_i \neq \alpha_{i-1}$:
    $i := i + 1$
    Create a colouring α_i of $V(\mathcal{G})$ such that $\forall u, v \in V(\mathcal{G})$:
        $\alpha_i(u) = \alpha_i(v) \Leftrightarrow$
        - $\alpha_{i-1}(u) = \alpha_{i-1}(v)$, and
        - u and v have identically coloured neighbourhoods in α_{i-1}.
EndWhile
return α_i.

Stable colourings

Definition

A colouring α of G is *stable* if for all $u, v \in V(G)$ it holds that:
if $\alpha(u) = \alpha(v)$, then u and v have identically coloured neighbourhoods.

Observation 3

By definition, the colour refinement procedure ends with a stable colouring.
(Regardless of the initial colouring)

Nomenclature

- A k-colouring α of $\mathcal{G}$ is *uniform* if $k = 1$.
- A k-colouring α of **graph** G is *discrete* if every colour appears exactly once. (So: $k = |V(G)|$, and for all $c \in \{1, \ldots, k\}$: $|\alpha^c| = 1$.)

- A k-colouring α of $\mathcal{G} = \{G, H\}$ is *balanced* if for every colour $c \in \{1, \ldots, k\}$, it holds that $|\alpha^c \cap V(G)| = |\alpha^c \cap V(H)|$.
- A k-colouring α of $\mathcal{G} = \{G, H\}$ *defines a bijection f* if for every colour $c \in \{1, \ldots, k\}$, it holds that $|\alpha^c \cap V(G)| = |\alpha^c \cap V(H)| = 1$.

  In this case, the bijection $f : V(G) \to V(H)$ is chosen such that for all $v \in V(G)$, $\alpha(v) = \alpha(f(v))$.

Stable & Unbalanced Colourings

Let α be the stable colouring of $\mathcal{G} = \{G, H\}$ from colour refinement, starting with the *uniform colouring*.

Observation

If α defines a bijection $f : V(G) \to V(H)$, then f is the unique *isomorphism from G to H.*

Proof: Inductively: any isomorphism g is colour preserving for the uniform colouring. Invariant $\Rightarrow$ g remains colour preserving in each refinement step. If α defines a bijection $f : V(G) \to V(H)$, must be true that $f = g$. $\square$

Similarly, we conclude that:

Observation

If α is unbalanced, then there is no isomorphism from G to H.

Q: But what if α is balanced, but does not define a bijection? (During group part.)

Partitions

Q: Does colour refinement always give the same colouring?

A: No, because we did not specify how the new colours should be chosen.
But ignoring the choice of colour (names), and only considering the resulting *partition* of the vertices, the result is indeed unique.

Basics on partitions:

- A *partition* π of V is a set of non-empty subsets of V, such that every element of V appears in exactly one subset.
- Elements of a partition (the subsets of V): also called *cells*.
- For $u, v \in V$ and a partition π of V, we write $u \equiv_\pi v$ to denote that u and v appear in the same cell of π. This is an *equivalence relation*. (think of $u \equiv_\pi v$: u and v have the same colour)

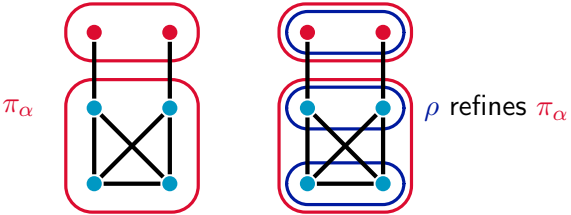
Partition Refinements

Definition

With a k-colouring α of $\mathcal{G}$, we associate the following partition of nodes into colour classes: $\pi_\alpha = \{\alpha^c \mid |\alpha^c| \geq 1\}$.

Definition

For two partitions π and ρ of V, we say that ρ refines π if for all $u, v \in V$:
$u \equiv_\rho v \implies u \equiv_\pi v$.
If in addition π is not equal to ρ, then ρ strictly refines π.



Coarsest Stable Partition

Definition

Partition π of $V(\mathcal{G})$ is stable *if colouring of the cells is a stable colouring.*

A stable partition π of $V(\mathcal{G})$ is a coarsest stable partition *if there is no stable partition ρ of $V(\mathcal{G})$ such that π strictly refines ρ.*

Analogously, for a given start partition π^0, we define the coarsest stable partition that refines π^0.

Theorem

For every partition π^0 of $V(\mathcal{G})$, there is a unique *coarsest stable partition of $V(\mathcal{G})$ that refines π^0, which is exactly the partition given by colour refinement, when starting with π^0.*

Coarsest Stable Partition

Theorem

For every partition π^0 of $V(\mathcal{G})$, there is a unique *coarsest stable partition of $V(\mathcal{G})$ that refines π^0, which is exactly the partition given by colour refinement, when starting with π^0.*

Proof sketch: For this proof, we ignore the colours assigned by colour refinement, and just consider the *corresponding partitions* $\pi^0, \ldots, \pi^\ell$.

Idea (induction step): let ρ be any partition that refines a partition π^i.
If colour refinement computes a partition π^{i+1} from π^i, then either ρ is unstable, or ρ also refines π^{i+1}. (Details: see Chapter 2 of the reader.)

So any stable partition ρ (that refines π^0) refines the final partition π^ℓ given by colour refinement. $\square$

Does this solve Graph Isomorphism?

What we learned so far: The Colour refinement algorithm applied to graph set $\{G, H\}$:

- results in a coarsest stable partition of the vertices.
- finds an isomorphism if the output is balanced, and discrete for both graphs.
- proves no isomorphism exists if the output is unbalanced.
- does not provide a definitive answer if the output is balanced but not discrete.

Using the Colour refinement algorithm we can partition a set of graphs $\{G_1, \ldots, G_\ell\}$ into equivalence classes:

- Two graphs G_i and G_j are equivalent if the output of Colour refinement is balanced.
  (Exercise: verify that this defines an equivalence relation on the set of all graphs.)

Symmetry: Automorphisms

Another application of colour refinement:

- An *automorphism* of a graph G is an isomorphism from G to *itself*.
- Every graph has at least one automorphism f, called the *trivial automorphism*: set $f(v) = v$ for all v.
- A graph on n vertices has at most $n!$ automorphisms (example: K_n).
- The *Automorphism Counting (#Aut) Problem* asks, given a graph G, how many automorphisms does G have?

Automorphism

Reformulating the previous observations, we can immediately conclude:

Observation

Let α be the coarsest stable colouring of G.

- *Any automorphism of G is colour preserving (with respect to α).*
- *If α is discrete, then G has only one automorphism.*

If α is not discrete, we again need more ideas to solve the #Aut problem...
(During the group part of the project.)

Project: This Week

- On Canvas: *instances* of the GI problem
  - Consist of *lists of graphs*.
- Outcome of the algorithm is a *partition into equivalence classes*.
- Task: implement the exact algorithm as presented today and in the reader. Verification through counting iterations and sizes of the equivalence classes (details on Canvas).
- The instances on canvas are similar to the ones used to test in Codegrade.
- Some of the instances are small enough to allow easy visualization / manual checking of your algorithms (remember the .dot file visualizers from last week).

Milestone:

- Week after the break: Everyone **individually** develops a Python program that implements the colour refinement algorithm as it was presented today.
  The output: list of sizes of the cells in the coarsed stable partition, flag if the result is discrete, count of the number of iterations.

Assignment/project notes

- There is a reader available on Canvas specifically created for this course.
- The assignment description specifies the output format.
- Two things to do to sign-off:
  1. Automated validation of your code using CodeGrade (available after this lecture).
  2. Do sign-off interview with TA/Teacher (after CodeGrade validation).
- Passing this assignment is mandatory to start the group phase of the project.
- If there are any issues (bugs/mistakes/typos/...) anywhere, please let us know.

Note: sign-off interview is stricter than last year(s):

- Three questions without access to code.
- Quick and correct answers to pass.
- Only one chance to repair in case of a fail.

Practical Python Notes

- The instances are given as text files, that can be read using the Python modules `graph_io.py` and `graphs.py`.
- For reading, e.g., the file `cref9vert_4_9.grl`, you can use:

```
with open('cref9vert_4_9.grl') as f:
    L=load_graph(f,read_list=True)
```

  Then L will be a *list of graphs*.
  So use `L[0]` to get the first graph in the list.
- For visualisation, recall that you can assign colours to vertices v by typing e.g.:

```
for v in G.vertices:
    v.color = colour_dict[v]
```

  and then writing the graph G to a `.graphML` file using

```
with open('colorful.graphML','w') as f:
    write_visualization(G,f)f
```

  Do **not** use `v.color` while running the algorithm!!

Some hints / stuff that went wrong in previous years

- To compare two graphs, their colouring needs to mean the same thing (e.g., "blue" refers to "one blue and two green neighbours" in both graphs). This can be achieved by colouring them at the same time.
- The pseudocode defines the colouring in terms of a property of pairs of vertices. That does not mean you need to check all pairs! You just need to ensure all vertices of the same colour had identically coloured neighbourhoods and had the same colour (in the previous iteration).
- A colouring can be represented in many ways. You can view it as a function $\alpha : V(G) \to \mathbb{Z}$, which is often easiest to represent using a dictionary (or multiple). Think about what is a good way for your algorithm.
- Count the iterations from **the uniform colouring** to the first time that you reach the coarsest stable colouring. Your algorithm will often need additional iterations to conclude that the colouring is stable.
- If your algorithm works correctly on "almost all instances", "most instances", "all instances except one", etc., you algorithm does not work correctly.

Finally

- Study/understand the algorithm before you start implementing it.
- Teachers and TAs are available to help/explain (during the sessions).
- For most students, working during the sessions is sufficient.
- The last session can be very crowded, if you can sign off early this is highly recommended (and appreciated).
- Don't forget: Complete history of all AI interactions needs to be submitted.

Good luck and have fun!